SolarPredict Final Year Project Defense Guide

1) Project Overview

SolarPredict is a full-stack software system that predicts short-term solar irradiance for two Cameroonian cities (Douala and Maroua) and provides microgrid operational recommendations.

It combines:

- Machine Learning (`XGBoost Model II`)
- Live weather data (`Open-Meteo API`)
- Backend API (`FastAPI`)
- Persistent storage (`SQLite`)
- Frontend dashboard (`React + Vite + Recharts`)

Core Objective

Deliver a practical decision-support tool for energy planning:

- Predict expected solar irradiance (`W/m²`)
- Translate predictions into actionable microgrid advice
- Visualize current and historical values for technical users

2) Problem Statement and Motivation

Many regions face unstable electricity supply and need better renewable integration.

Solar output is weather-dependent; operators need near-term predictions and clear operational guidance.

SolarPredict addresses this by:

- Forecasting irradiance from meteorological variables
- Classifying energy availability level (High / Moderate / Low)
- Recommending immediate battery/grid strategy

3) High-Level System Architecture

Data Flow

1. User selects city and time horizon in the frontend.
2. Frontend calls FastAPI endpoints.
3. Backend requests weather forecast from Open-Meteo.
4. Backend engineers Model II features.

5. Backend runs XGBoost prediction.
6. Backend computes microgrid recommendation and optimization outputs.
7. Backend saves prediction records to SQLite.
8. Frontend renders cards, charts, tables, and recommendations.

Layered Design (Clean Architecture Style)

- **Presentation layer**: React pages/components
- **Application/API layer**: FastAPI routes + schema validation
- **Domain logic layer**: feature engineering + recommendation + optimization logic
- **Infrastructure layer**: weather API client + SQLite persistence + model file loading

4) Complete Folder Structure (Explained)

```
SolarPredict/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ __init__.py
■ ■ ■■■ config.py
■ ■ ■■■ database.py
■ ■ ■■■ features.py
■ ■ ■■■ main.py
■ ■ ■■■ microgrid.py
■ ■ ■■■ model_loader.py
■ ■ ■■■ schemas.py
■ ■ ■■■ weather.py
■ ■ ■■■ services/
■ ■ ■■■ microgrid_recommendation.py
■ ■■■ inspect_db.py
■ ■■■ requirements.txt
■ ■■■ solar_predict.db
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ api/client.js
■ ■ ■■■ components/
■ ■ ■ ■■■ Layout.jsx
■ ■ ■ ■■■ RecommendationBadge.jsx
■ ■ ■ ■■■ StatCard.jsx
■ ■ ■■■ pages/
■ ■ ■ ■■■ About.jsx
■ ■ ■ ■■■ Dashboard.jsx
■ ■ ■ ■■■ History.jsx
■ ■ ■ ■■■ Microgrid.jsx
■ ■ ■ ■■■ Prediction.jsx
■ ■ ■■■ styles/index.css
■ ■ ■■■ App.jsx
■ ■ ■■■ main.jsx
■ ■■■ index.html
■ ■■■ package.json
■ ■■■ package-lock.json
■ ■■■ vite.config.js
■■■ docs/
■ ■■■ SolarPredict_FYP_Defense_Guide.md
```

```
■■■ Model/
■ ■■■ douala_xgb_model_ii.pkl
■ ■■■ maroua_xgb_model_ii.pkl
■■■ Data/
■■■ training/
■■■ feature_model_II.ipynb
■■■ Feature_engineering.ipynb
■■■ README.md
```

What each top-level folder does

- `backend/`: API, model inference, DB access, business logic
- `frontend/`: user interface and charts
- `Model/`: trained model artifacts
- `Data/` and `training/`: dataset and model training notebooks
- `docs/`: project documentation for report/defense

5) Backend Code Structure (File-by-File)

`app/main.py` (API Entry Point)

Responsibilities:

- Create FastAPI app
- Configure CORS for frontend origin
- Initialize DB and preload city models on startup
- Expose all API routes

Endpoints:

- `GET /` basic API info
- `GET /api/health` health check and supported cities
- `GET /api/cities` city metadata
- `GET /api/dashboard?city=...` live weather + prediction + recommendation + aggregate stats
- `POST /api/predict` multi-hour forecast
- `GET /api/history` SQLite history
- `GET /api/microgrid/recommendation` rule lookup by prediction value
- `POST /api/microgrid/optimize` full microgrid simulation

`app/config.py` (Configuration)

- Defines absolute paths (`BASE_DIR`, `MODEL_DIR`, `DB_PATH`)
- Defines city metadata:

- Douala (lat/lon/timezone/model file)
- Maroua (lat/lon/timezone/model file)

- Defines strict ordered feature list (`MODEL_II_FEATURES`) to match training feature order

`app/weather.py` (External Weather Client)

- Calls Open-Meteo forecast API asynchronously with `httpx`
- Retrieves hourly:

- `temperature_2m`
- `relative_humidity_2m`
- `wind_speed_10m`
- `precipitation`

- Converts each hourly weather record into engineered Model II feature row
- Returns normalized records used by prediction and dashboard flows

`app/features.py` (Feature Engineering)

Builds the exact 21 features expected by Model II:

- Time/calendar: `YEAR`, `MO`, `DY`, `HR`
- Weather: `T2M`, `RH2M`, `PRECTOTCORR`, `WS10M`
- Cyclic encodings: `hour_sin/cos`, `month_sin/cos`, `day_sin/cos`
- Extra temporal: `day_of_year`
- Binary: `is_daytime`, `is_rainy`
- Transformed: `rain_log1p`, `ws10m_log1p`
- Seasonal flags: `season_dry`, `season_rainy`

The function `build_feature_dataframe` guarantees prediction column ordering correctness.

`app/model_loader.py` (Model Inference Layer)

- Loads each city-specific `.pkl` model only once (in-memory cache)
- Validates file existence and city id
- Runs model prediction and clamps negatives to zero:

- irradiance cannot be physically negative

`app/services/microgrid_recommendation.py` (Rule Engine)

Pure decision logic:

- `prediction > 700` -> `High Solar Energy`
- `400 <= prediction <= 700` -> `Moderate Solar Energy`

- `prediction < 400` -> `Low Solar Energy`

Recommendation texts map exactly to your requirement.

`app/microgrid.py` (Optimization/Simulation Engine)

Computes hourly microgrid operation:

- Convert irradiance to energy generated (`kWh`) from panel area and efficiency
- Compare production against hourly load
- If surplus: charge battery, then export remainder
- If deficit: discharge battery, then import from grid
- Accumulate totals:

- total predicted solar
- total grid import/export
- battery use
- self-sufficiency percentage

Also includes current microgrid recommendation for operator clarity.

`app/database.py` (SQLite Access Layer)

- Creates and migrates `predictions` table to required schema
- Supports migration from legacy names (old columns)
- Inserts prediction records
- Reads history and analytics summaries
- Provides dashboard aggregates:

- total predictions
- per-city counts
- latest rows
- average prediction by city

`app/schemas.py` (Validation Contracts)

Pydantic models enforce API payload contracts:

- Request schema (`PredictionRequest`, `MicrogridRequest`)
- Response schema (`PredictionResponse`, `DashboardResponse`, `MicrogridResponse`)
- Nested typed objects for weather snapshots and schedule rows

This provides strong correctness and self-documenting API docs (`/docs`).

`backend/inspect_db.py`

Utility script to inspect current SQLite tables, columns, and row counts.

Useful for demos and debugging.

6) Frontend Code Structure (File-by-File)

`src/main.jsx`

React entry point:

- Mounts app to DOM root
- Wraps app in `BrowserRouter` for client-side routes
- Loads global CSS

`src/App.jsx`

Central route map:

- `/` -> Dashboard
- `/prediction` -> Prediction page
- `/history` -> History page
- `/microgrid` -> Microgrid page
- `/about` -> About page

`src/components/Layout.jsx`

Shell layout:

- Sidebar navigation
- Branding
- Routed content area (`Outlet`)

Promotes consistent UI and reusable navigation.

`src/components/StatCard.jsx`

Reusable metric display card:

- Label + value + unit
- Optional icon
- Variant styles (default/accent)

Used across dashboard and microgrid pages.

`src/components/RecommendationBadge.jsx`

Displays status and textual recommendation with semantic color theme:

- Green -> high
- Amber -> moderate
- Red -> low

`src/api/client.js`

Single API integration module:

- Centralizes all HTTP requests
- Handles backend connection and error formatting
- Exposes typed operations:

- `health`, `cities`
- `dashboard`, `predict`, `history`
- `microgridRecommendation`, `optimizeMicrogrid`

`src/pages/Dashboard.jsx`

Main operational view:

- City selector
- 6 required cards:

- Current Temperature
- Humidity
- Wind Speed
- Precipitation
- Predicted Solar Irradiance
- Microgrid Recommendation

- Last updated and total predictions info
- Historical chart and latest predictions table

`src/pages/Prediction.jsx`

Forecast execution interface:

- User selects city and horizon
- Calls prediction API
- Shows line chart over time
- Shows recommendation for current prediction
- Displays table of hourly results

`src/pages/History.jsx`

Database browsing UI:

- Optional city filtering
- Shows stored records from SQLite with full required schema fields

`src/pages/Microgrid.jsx`

Simulation input + result visualization:

- Inputs: battery capacity, load, panel area
- Summary KPIs cards (solar, import, export, self-sufficiency)
- Hourly energy balance chart
- Schedule table with status/recommendation

`src/pages/About.jsx`

Static project explanation:

- Objective
- Tech stack
- Pipeline summary
- Microgrid rule summary

`vite.config.js`

Development proxy:

- Frontend runs on `5173`
- `/api` proxied to backend `http://127.0.0.1:8000`

This avoids CORS complexity during local dev.

7) Database Design and Relations

Main Required Table: `predictions`

Columns:

- `id` (INTEGER, PK)
- `city` (TEXT)
- `temperature` (REAL)
- `humidity` (REAL)
- `wind_speed` (REAL)

- `precipitation` (REAL)
- `prediction` (REAL)
- `timestamp` (TEXT ISO datetime)

Existing Additional Table

`microgrid_runs` may exist from earlier iterations. It can be kept for historical simulation records or removed if not needed in final scope.

Relations

There are currently no explicit foreign keys.

The design is simple and suitable for prototype-scale deployment.

For future work:

- Add `cities` master table
- Reference `city_id` in `predictions` and `microgrid_runs`

8) Functionalities Implemented

1. City-aware weather retrieval (Douala/Maroua)
2. Model II feature engineering with seasonal logic
3. XGBoost inference for irradiance forecasting
4. Rule-based microgrid recommendation
5. Dashboard live cards and recommendation visualization
6. Historical storage in SQLite
7. Multi-hour prediction view with charts/tables
8. Microgrid optimization simulation with battery/grid balancing
9. Validation + interactive API docs via FastAPI/Pydantic
10. Frontend-backend integration with robust error handling

9) How to Run (Demonstration Checklist)

Backend

```
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
```

Verify:

- `http://127.0.0.1:8000/docs`

- `http://127.0.0.1:8000/api/health`

Frontend

```
cd frontend
npm install
npm run dev
```

Open:

- `http://localhost:5173`

Important:

- Keep backend and frontend terminals running simultaneously.

10) Testing Strategy (What to Test Before Defense)

A) API Functional Tests

1. `GET /api/health`

- Expect `status: ok`

2. `GET /api/dashboard?city=douala`

- Expect all card fields + recommendation

3. `POST /api/predict` with valid body

- Expect list of predictions with nested weather + microgrid advice

4. `GET /api/history`

- Confirm saved rows and filters

5. `POST /api/microgrid/optimize`

- Confirm schedule length and KPI totals

B) Validation/Negative Tests

- Invalid city -> 400
- `hours` out of range -> validation error
- Missing model file -> backend 500 with clear message
- Backend stopped -> frontend shows meaningful connection error

C) Rule Boundary Tests (Critical)

- Prediction = `701` -> High
- Prediction = `700` -> Moderate
- Prediction = `400` -> Moderate

- Prediction = `399.99` -> Low

D) UI Tests

- Navigation route correctness
- Charts render with returned data
- Cards update when city changes
- History filters correctly
- Microgrid KPIs update when parameters change

E) Persistence Tests

- Run dashboard/predict
- Confirm rows appear in `predictions`
- Confirm schema columns match required names

11) Defense Talking Points (Presentation Script)

Suggested Demo Flow (8-12 minutes)

1. Introduce energy reliability problem and need for forecasting.
2. Show architecture diagram and data flow.
3. Show feature engineering logic and why Model II avoids leakage.
4. Run live dashboard for Douala and explain six cards.
5. Run prediction for 24h and explain chart/table.
6. Show microgrid optimization and explain import/export logic.
7. Show history table and database persistence.
8. Discuss validation, testing, and future improvements.

Why this project is strong

- Real-time external data integration
- End-to-end ML deployment, not only model training
- Practical decision support layer for operators
- Full-stack implementation with observable outputs
- Clear extension path to production-grade system

12) Anticipated Viva Questions and Model Answers

Q1: Why XGBoost Model II?

Because it gives strong tabular regression performance, fast inference, and handles nonlinear weather relationships well.

Q2: Why these features?

They capture meteorological drivers and temporal seasonality. Cyclic encodings preserve periodic nature of hour/month/day.

Q3: Why avoid solar leakage variables?

Using future or target-like solar variables in inputs would produce unrealistic over-optimistic predictions.

Q4: Why FastAPI?

Fast development speed, automatic OpenAPI docs, async capability for external API calls, and good validation with Pydantic.

Q5: How does the microgrid optimizer work?

It calculates hourly energy balance: solar production vs load; then battery charge/discharge; then grid import/export if needed.

Q6: What are your limits?

Single weather provider, simple battery assumptions, no degradation/loss model, no uncertainty intervals.

Q7: How to improve?

Add uncertainty quantification, multi-city scaling, auth, task queue, PostgreSQL, and cloud deployment.

13) Known Constraints and Future Work

Current constraints:

- External API dependency on Open-Meteo uptime
- Rule-based recommendation is deterministic (no economic optimization)
- SQLite limits concurrent write scalability

Planned improvements:

- Add probabilistic forecasting intervals
- Add tariff-aware optimization objective
- Add user authentication and role-based dashboard
- Move to PostgreSQL + background workers
- Containerize with Docker and CI/CD pipeline

14) Conclusion

SolarPredict demonstrates a complete, practical, and defendable final-year project:

- data ingestion,
- feature engineering,
- ML inference,
- optimization logic,
- persistence,
- and full user interface.

It is a strong foundation for real-world renewable-energy decision support systems in Cameroon and similar regions.